

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS:

Farfán Rodríguez Jorge Emmanuel

PARALELO:

"A"

SEMESTRE:

Quinto

ACTIVIDAD:

Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

Enlace del Repositorio GitHub: https://github.com/emmanuel-6193/Desarrollo_de_SI.git

PERIODO ABRIL 2026 - AGOSTO 2026



Actividades a desarrollar en la Unidad #1

Actividad: Estudio de Caso

Sistema de Gestión de Incidentes (Help Desk)

En este documento presento el desarrollo de mi actividad, enfocada en el diseño de un Sistema de Gestión de Incidentes (Help Desk). Para resolver los requerimientos de la unidad, elaboré los diagramas UML de Casos de Uso y de Clases que estructuran el sistema, y decidí aplicar los patrones Singleton, Factory Method y Observer para solucionar la lógica de creación y notificación de los tickets. Por último, integré esta solución bajo la arquitectura MVC.

1. Modelado UML y el Flujo Dinámico del Sistema

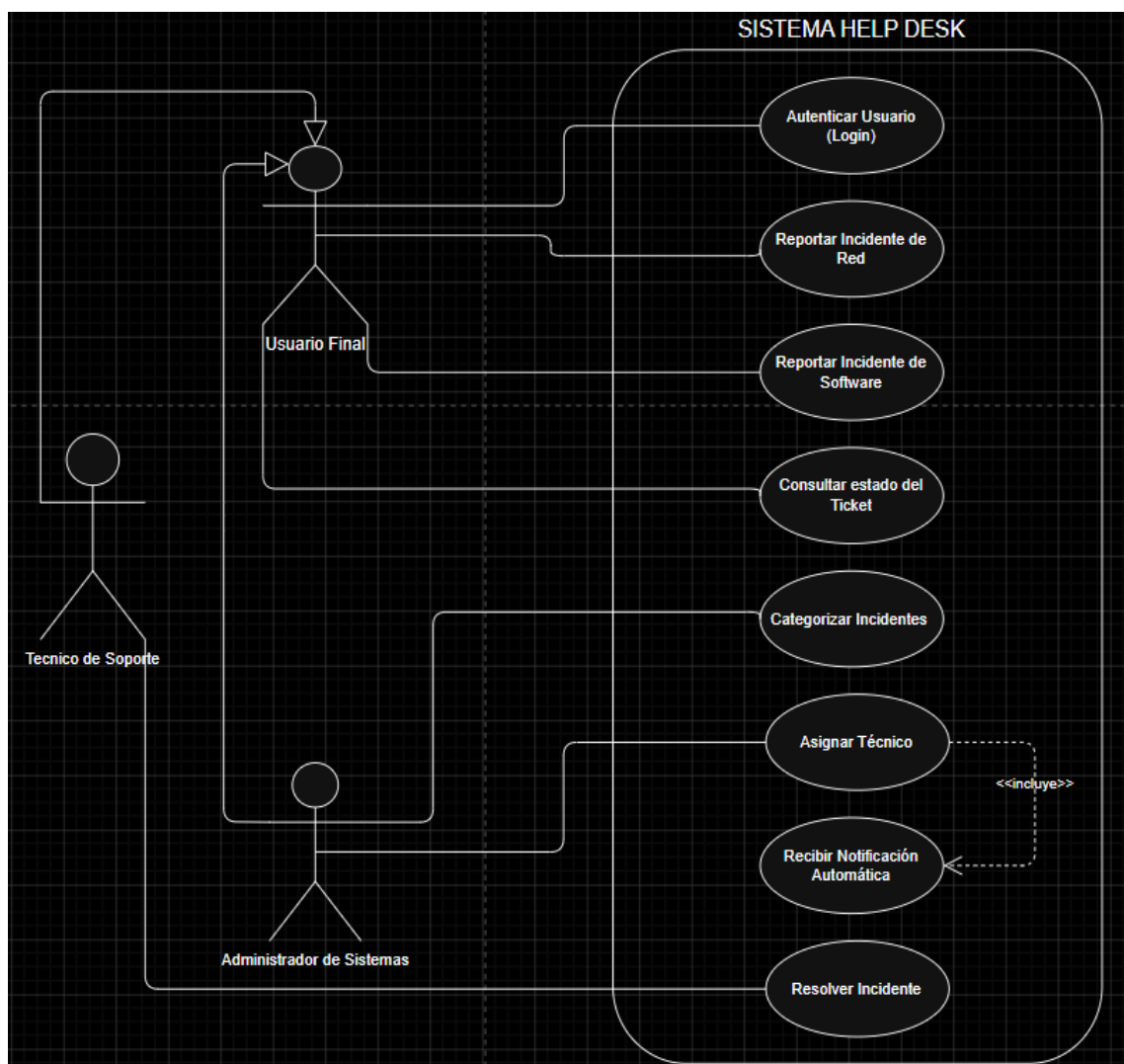
Para abarcar el sistema desde todas sus perspectivas (quién lo usa, cómo se estructura internamente y cómo se comporta en la línea de tiempo), desarrollé los siguientes tres diagramas:

1.1. Diagrama de Casos de Uso

Al sentarme a diseñar mi primer impulso fue crear casos de uso separados para cada tipo de usuario. Sin embargo, al analizar los requerimientos, noté que esto generaría una redundancia masiva. Por ello, decidí aplicar el concepto de Generalización de Actores (herencia):

- Usuario Final: Lo definí como mi actor base. Sus permisos están restringidos estrictamente a dos acciones fundamentales: ingresar datos (reportar incidentes) y realizar consultas de solo lectura (ver el estado de sus tickets). No tiene acceso a ninguna lógica de negocio interna.
- Técnico de Soporte: Al reflexionar sobre este rol, me di cuenta de que un técnico también es un empleado. Si su propia computadora falla, necesita reportarlo. Por lo tanto, mediante la herencia, hice que adquiriera automáticamente todas las facultades del Usuario Final. A esto le sumé sus casos de uso específicos: visualizar su bandeja de tareas, gestionar la resolución y cerrar los incidentes que se le asignen.

- Administrador de Sistemas: Representa el perfil de mayor jerarquía. Hereda del perfil base, pero es el único actor para el cual diseñé permisos de escritura sobre la carga de trabajo de otros, permitiéndole categorizar problemas y distribuir los tickets manualmente.
- Lógica de Inclusión: Quería evitar a toda costa el error humano. En muchos sistemas, un administrador asigna un ticket, pero olvida avisarle al técnico, lo que retrasa la atención. Para solucionarlo desde el diseño, establecí una relación de dependencia estricta desde “Asignar Técnico” hacia “Recibir Notificación Automática”. Con esto, garantizo que, a nivel de requerimientos, una asignación jamás se dará por completada en la base de datos si el sistema no logra disparar exitosamente la alerta al empleado correspondiente.

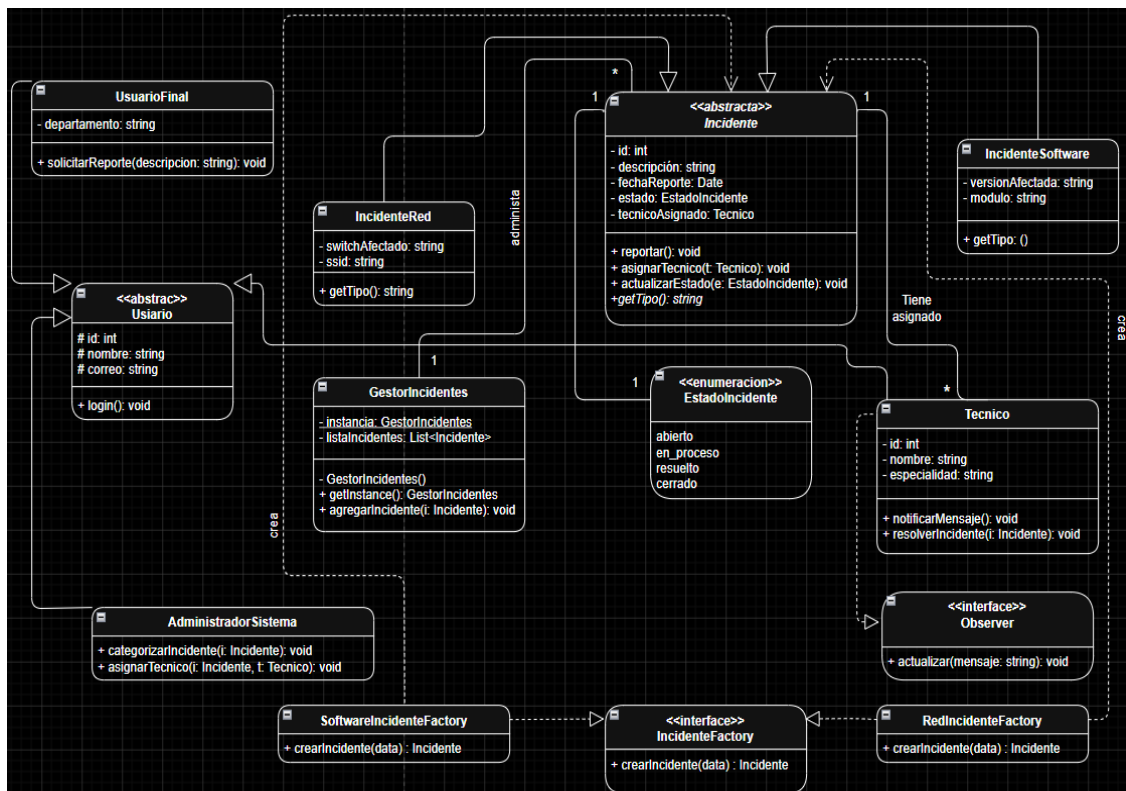


1.2. Diagrama de Clases UML

Mi prioridad fue garantizar un bajo nivel de acoplamiento y un alto nivel de cohesión. No quería un sistema donde todas las clases dependieran directamente unas de otras.

Abstracción y Encapsulamiento: Decidí crear una superclase abstracta llamada Usuario. Comprendí que en el mundo real no tiene sentido instanciar un usuario genérico; siempre será un Cliente, un Técnico o un Administrador. Por ello, dejé Usuario como abstracta para concentrar los atributos comunes (ID, nombre, correo, credenciales) y obligué a que las clases hijas definan sus propios métodos especializados. Además, todos los atributos son private para proteger los datos, accediendo a ellos únicamente a través de métodos Getters y Setters.

Uso de Interfaces: En lugar de conectar la lógica de creación y notificación directamente a mis clases principales, estructuré el diagrama utilizando contratos o interfaces (Observer y FabricaIncidentes). Esto me asegura que, si en un futuro necesito reescribir por completo el código de la clase Tecnico, el núcleo de mi sistema (el Gestor de Incidentes) no se va a romper ni requerirá modificaciones, ya que solo interactúa con la interfaz abstracta.



1.3. Diagrama de Secuencia

Mientras que el diagrama de clases me mostraba qué entidades existían, necesitaba visualizar cómo iban a interactuar en la memoria RAM con el paso de los segundos. Por ello, tracé el ciclo de vida completo de un requerimiento paso a paso:

El Usuario envía su formulario de reporte desde la interfaz gráfica.

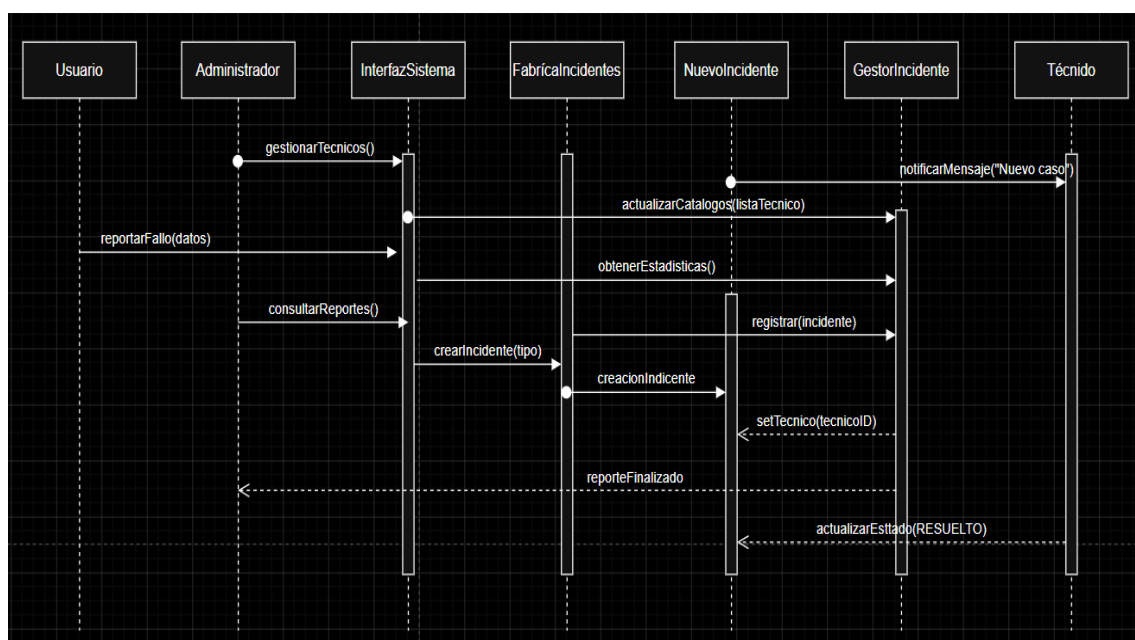
El sistema atrapa esos datos y, en tiempo de ejecución, delega la creación del ticket a la FabricaIncidentes para aislar la complejidad.

Una vez que el objeto del ticket existe, viaja hacia el GestorIncidentes (que opera como el corazón del sistema), donde es validado y encolado en la memoria central.

Posteriormente, el Administrador interactúa con el gestor para revisar la cola de tickets y asignar uno a un técnico específico.

Inmediatamente después de la asignación, diseñé el disparo de un mensaje asíncrono (una notificación) que viaja hacia la línea de vida del Técnico.

Con este esquema dinámico, logré demostrar que mi flujo de datos tiene una trazabilidad completa, asegurando que la información no se pierde entre capas y que no existen cuellos de botella.





2. Análisis y Justificación de los Patrones de Diseño

Al estructurar la lógica interna y pensar en cómo se comportaría el código en producción, identifiqué tres grandes vulnerabilidades arquitectónicas. Para mitigarlas con estándares profesionales de la industria, estudié e implementé los siguientes patrones de diseño:

A. Patrón Creacional: Factory Method

- El Problema que identifiqué: Un ticket de Red tiene requerimientos muy técnicos; el sistema necesita validar la *dirección IP*, la *latencia (ping)* y la *topología*. Por otro lado, un ticket de Software necesita capturar la *versión del sistema operativo*, los *registros de error (logs)* y el *nombre del programa que falla*. Si yo intentaba programar toda esta lógica de validación y creación en un solo controlador gigante lleno de condicionales if/else, iba a terminar con un código extremadamente frágil y difícil de leer.
- Mi Solución: Para evitar esto, implementé el patrón Factory Method. Creé la interfaz *FabricaIncidentes*. Ahora, el núcleo de mi sistema no tiene ni idea de cómo se crea un ticket; simplemente delega la tarea a las clases expertas *RedIncidenteFactory* o *SoftwareIncidenteFactory*. Cada fábrica es una especialista autónoma que sabe cómo solicitar datos, validarlos y ensamblar su propio tipo de ticket.
- Valor Agregado: Esta decisión hace que mi proyecto sea a prueba de futuro. Si el próximo semestre la universidad (o el cliente) me pide añadir un módulo para “Incidentes de Hardware” o “Mantenimiento de Servidores”, no tendré que tocar ni romper el código que ya funciona. Simplemente programaré una nueva clase fábrica, cumpliendo a la perfección con el principio de que mi software debe estar abierto a la extensión, pero cerrado a la modificación.

B. Patrón Creacional: Singleton

- El Problema que identifiqué: En un Help Desk, hay múltiples actores operando al mismo tiempo. Al pensar en la memoria RAM del servidor, noté un riesgo grave: las condiciones de carrera. Si mi código permitía que cualquier controlador creara varias instancias mediante `new GestorIncidentes()`, los datos se fragmentarían. Un administrador vería una lista de 5 tickets en su espacio de memoria, mientras otro administrador vería una lista vacía. Peor aún, al generar números de seguimiento, ambos gestores podrían asignarle el Ticket #001 a dos problemas diferentes, corrompiendo la integridad de la base de datos de manera irreversible.



- **Mi Solución:** Para blindar mi sistema, diseñé mi clase GestorIncidentes aplicando el patrón Singleton. Lo primero que hice fue ocultar su constructor (haciéndolo `private`), impidiendo que nadie pueda crear copias de la clase. A cambio, creé un método público estático llamado `getInstance()`. Este método revisa la memoria: si el gestor no existe, lo crea una única vez; si ya existe, devuelve exactamente la misma referencia.
- **Valor Agregado:** Con este cerrojo arquitectónico, garantizo estrictamente que todas las pantallas, todos los usuarios y todos los módulos de mi aplicación lean y escriban sobre la misma y única fuente de la verdad en la memoria del servidor. La concurrencia queda bajo control total.

C. Patrón de Comportamiento: Observer

- **El Problema** que identifiqué: Al llegar al punto de la notificación, me pregunté: ¿Cómo sabe la pantalla del Técnico que el Administrador le acaba de asignar un ticket? La salida fácil era usar la técnica de Polling, es decir, programar la pantalla del técnico con un bucle infinito para que consulte a la base de datos cada 3 segundos preguntando: ¿Tengo tareas nuevas? Si la empresa tiene 50 técnicos conectados 8 horas al día, esto generaría decenas de miles de consultas inútiles, saturando el ancho de banda, sobrecargando el procesador del servidor y haciendo que todo el sistema se vuelva lento.
- **Mi Solución:** Decidí invertir el flujo de control aplicando el patrón Observer. Hice que mi GestorIncidentes actúe como el *Sujeto* (el que tiene la información) y los Técnicos como los *Observadores*. Ahora, el técnico simplemente se suscribe al sistema cuando inicia sesión y su computadora se queda inactiva, sin consumir red. En el milisegundo exacto en que el Administrador presiona Asignar, mi Gestor revisa su lista de suscritos y dispara el método `notificar()`, empujando la alerta directamente a la pantalla del técnico afectado.
- **Valor Agregado:** Transformé un sistema pesado, ineficiente y basado en la extracción constante, en un ecosistema reactivo y asíncrono. Esto no solo ahorra enormes cantidades de recursos del servidor, sino que brinda una experiencia de usuario instantánea y profesional.

3. Principios y Arquitectura Aplicada

Para garantizar que el código final de mi proyecto no solo funcione, sino que sea considerado código limpio y sea fácilmente comprensible para cualquier otro desarrollador que trabaje conmigo en el futuro, me aseguré de basar mis decisiones en los siguientes fundamentos arquitectónicos:



3.1. Cumplimiento de Principios SOLID

A lo largo de todo mi diseño, fui sumamente riguroso con el Principio de Responsabilidad Única (SRP). Al revisar mis diagramas, comprobé que ninguna clase hace el trabajo de otra:

- Las *Fábricas* se dedican única y exclusivamente a crear objetos.
- El *Gestor* se dedica únicamente a encolar datos y asignar técnicos.
- Las *Vistas* tienen la única responsabilidad de mostrar información (HTML/UI).

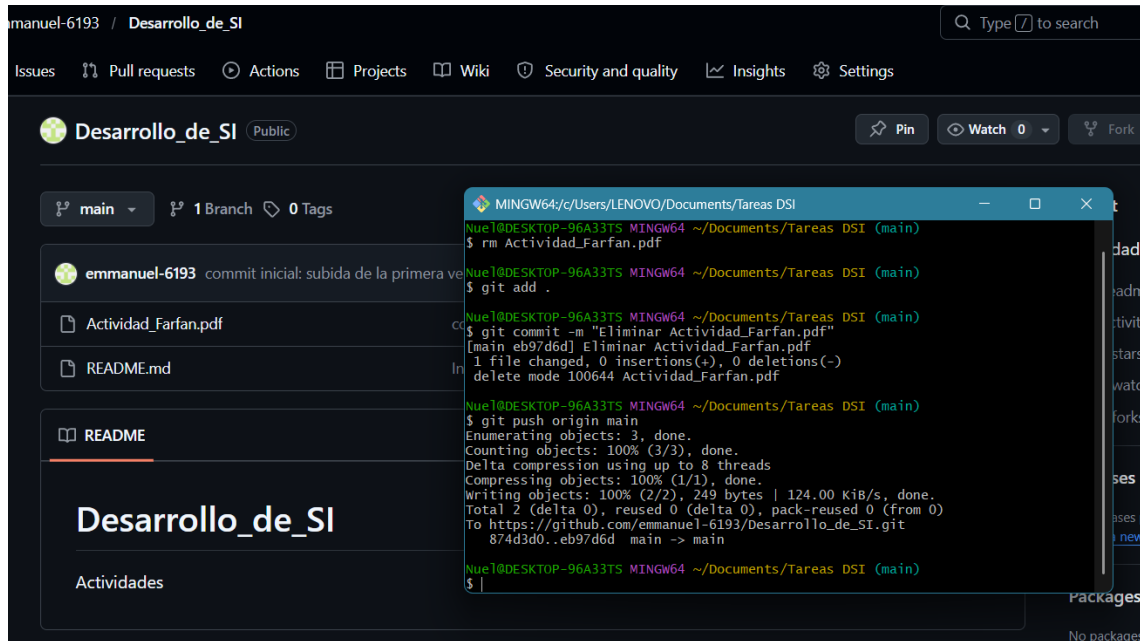
Esta segmentación me otorga una enorme tranquilidad. Si el día de mañana ocurre un error crítico al guardar un ticket, sé exactamente a qué módulo de mi código debo dirigirme (el Modelo/Gestor), sin tener la necesidad de perder el tiempo buscando errores dentro del código de la interfaz gráfica o de las alertas.

3.2. Arquitectura Modelo-Vista-Controlador

Para darle una estructura final unificada a todo el ecosistema, orquesté mi proyecto bajo el prestigioso paradigma MVC, separando físicamente mi sistema en tres grandes capas lógicas:

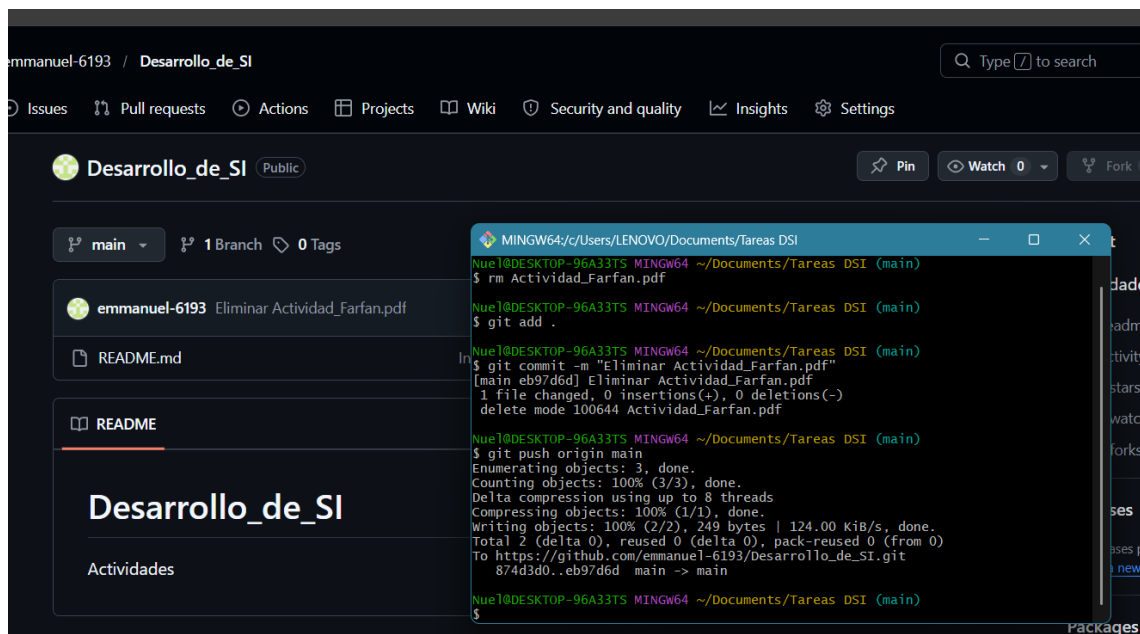
- El Modelo: Aquí encapsulé todas mis entidades de negocio (Usuario, Incidente y mi Singleton GestorIncidentes). Es el único lugar donde existen las reglas estrictas de la empresa y la conexión a los datos persistentes.
- La Vista: Representa las pantallas donde interactúan los humanos. Se divide en el portal amigable para que el Usuario Final haga sus reportes, el dashboard estadístico para el Administrador, y la bandeja de entrada para el Técnico. No contiene ni una sola línea de lógica de programación pesada.
- El Controlador: Es la capa intermedia que atrapa los eventos de la Vista (por ejemplo, el clic en el botón Enviar Reporte). Una vez que atrapa la petición, utiliza mis Fábricas para instanciar la memoria, le dice al modelo que almacene la información de forma segura mediante el Singleton, y finalmente dispara el *Observer* para avisar al técnico.

Evidencias de la Creación de las diferentes Ramas



```
MINGW64~/c/Users/LENOVO/Documents/Tareas DSI
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ rm Actividad_Farfan.pdf
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ git add .
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ git commit -m "Eliminar Actividad_Farfan.pdf"
[main eb97d6d] Eliminar Actividad_Farfan.pdf
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 Actividad_Farfan.pdf
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 249 bytes | 124.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/emmanuel-6193/Desarrollo_de_SI.git
874d3d0..eb97d6d  main -> main
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$
```

Eliminé el archivo en blanco que realizamos en la configuración del Git con el comando **rm Actividad_Farfan.pdf**



```
MINGW64~/c/Users/LENOVO/Documents/tareas DSI
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ rm Actividad_Farfan.pdf
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ git add .
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ git commit -m "Eliminar Actividad_Farfan.pdf"
[main eb97d6d] Eliminar Actividad_Farfan.pdf
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 Actividad_Farfan.pdf
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 249 bytes | 124.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/emmanuel-6193/Desarrollo_de_SI.git
874d3d0..eb97d6d  main -> main
Nue1@DESKTOP-96A33TS MINGW64 ~/Documents/Tareas DSI (main)
$
```

Con el comando **git push origin main** quedó correctamente eliminado y listo para crear las ramas solicitadas

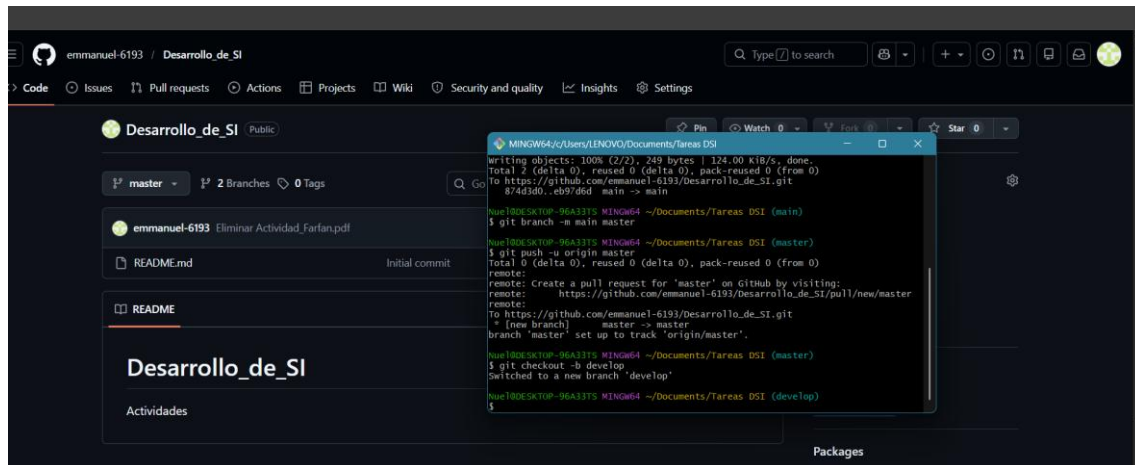


```
MINGW64~/Users/LENOVO/Documents/Tareas DSI
$ git add .
$ git commit -m "Eliminar Actividad_Farfan.pdf"
$ git push origin main
$ git branch -m main master
```

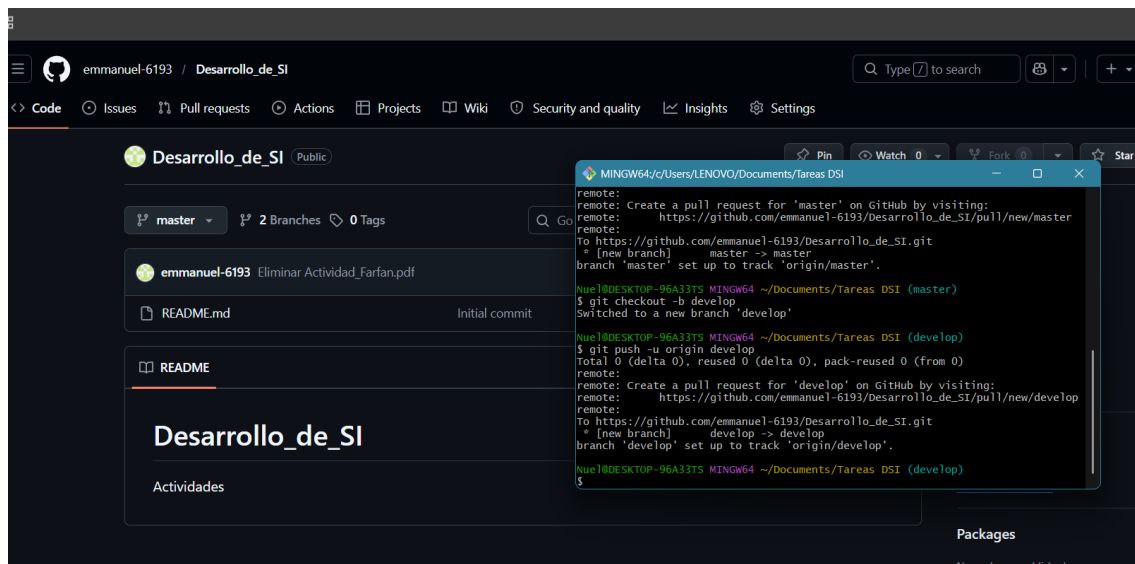
Ingreso el comando **git branch -m main master** para empezar el proceso de cambiar de main a master

```
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 249 bytes | 124.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/emmanuel-6193/Desarrollo_de_SI.git
874d3d0..eb97d6d main -> main
$ git push -u origin master
remote: Create a pull request for 'master' on GitHub by visiting:
remote: https://github.com/emmanuel-6193/Desarrollo_de_SI/pull/new/master
branch 'master' set up to track 'origin/master'.
```

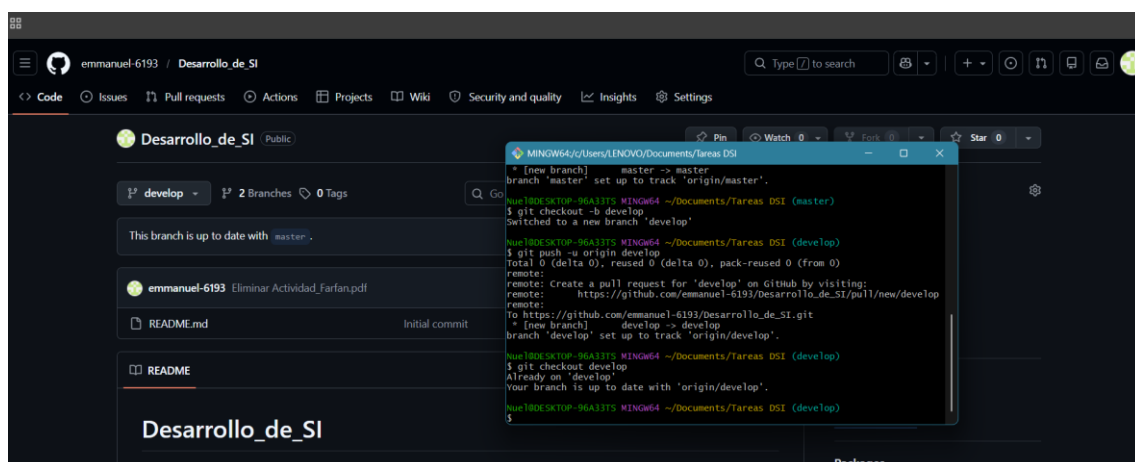
Cambiar de main a master ingreso el comando **git push -u origin master**



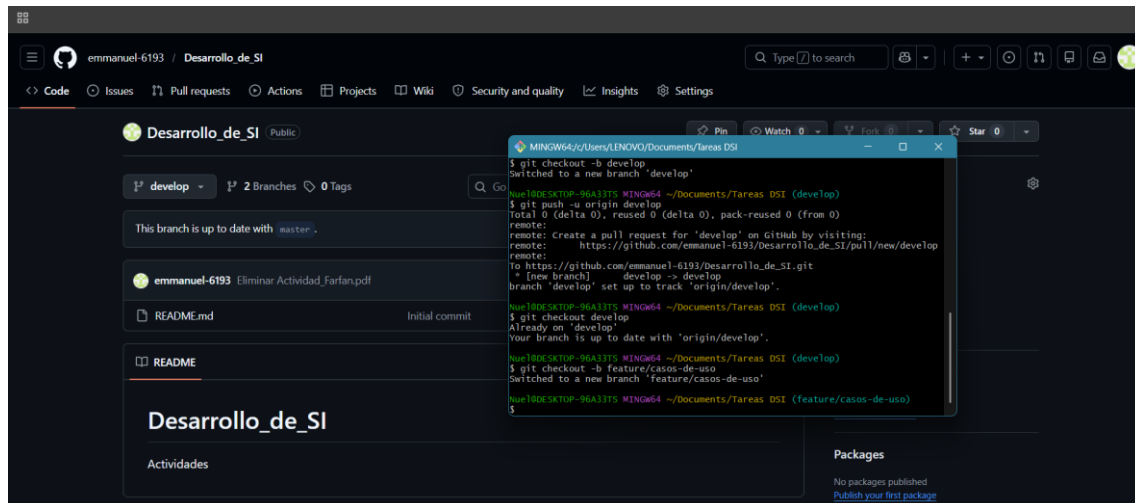
Hago la creación de rama develop con el comando **git checkout -b develop**



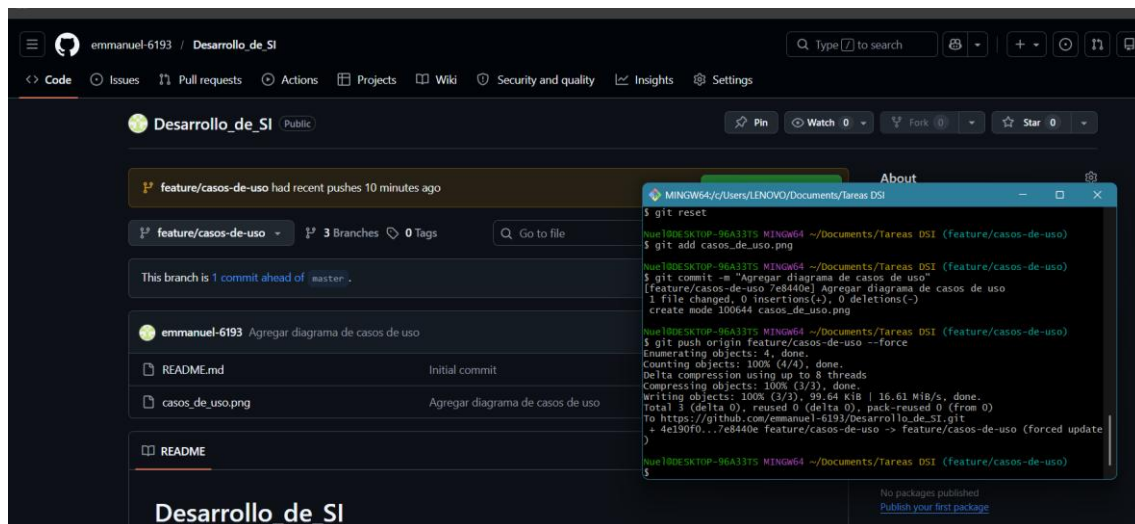
Correctamente configurada la rama develop con el comando **git push origin develop**



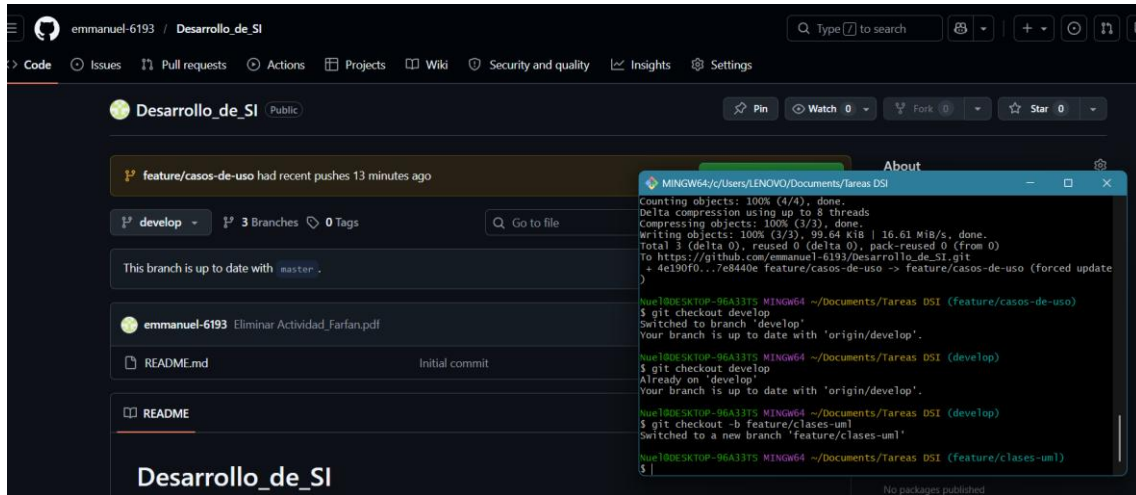
Me cambio a develop para crear la rama feature con el comando **git checkout develop**



Creo la rama feature/casos-de-uso con el comando **git checkout -b feature/casos-de-uso**

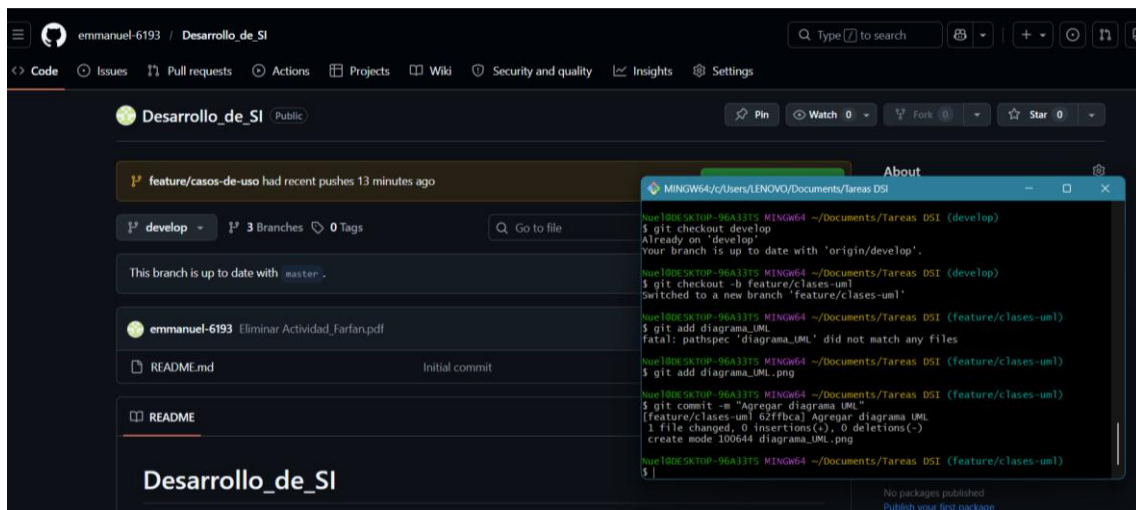


Subí por error los 3 diagramas a la misma rama pero soluciono el error con el comando **git push origin feature/casos-de-uso --force**



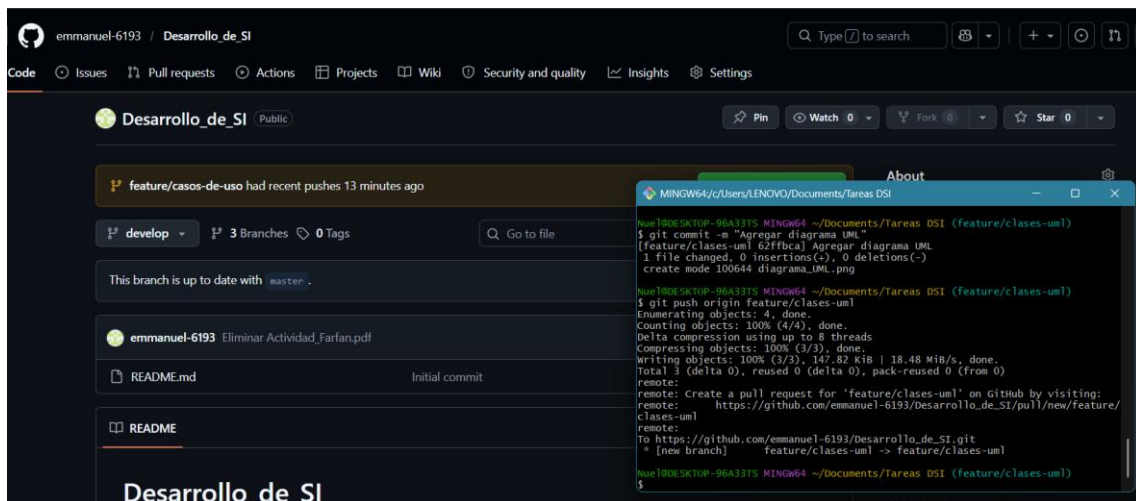
```
MINGW64/c/Users/LENOVO/Documents/Tareas DSI
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
$ git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.
$ git checkout -b feature/clases-uml
Switched to a new branch 'feature/clases-uml'
$
```

Creo la rama clases-uml con el comando **git checkout -b feature/clases-uml**



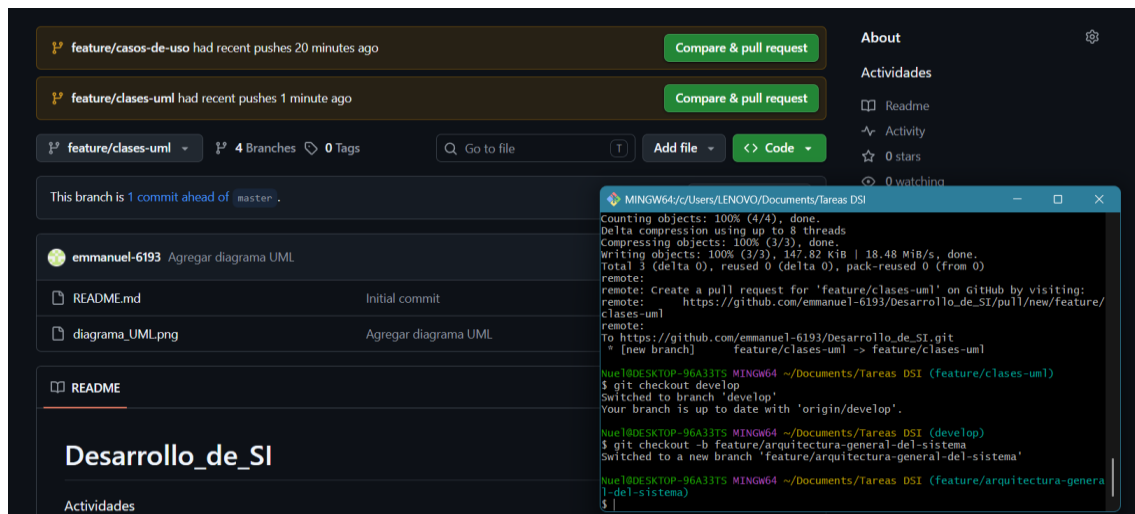
```
MINGW64/c/Users/LENOVO/Documents/Tareas DSI (feature/clases-uml)
$ git add diagrama_UML
fatal: pathspec 'diagrama_UML' did not match any files
$ git add diagrama_UML.png
$
```

Agrego el diagrama UML con el comando **git add diagrama_UML.png**

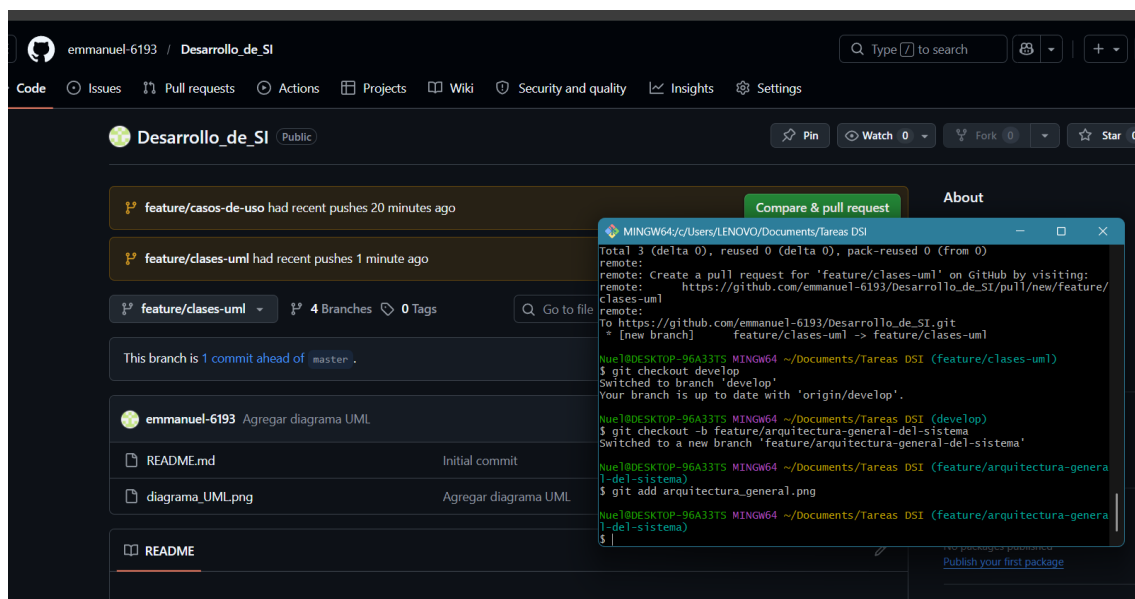


```
MINGW64/c/Users/LENOVO/Documents/Tareas DSI (feature/clases-uml)
$ git commit -m "Agregar diagrama UML"
[feature/clases-uml 62ffbc4] Agregar diagrama UML
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 diagrama_UML.png
$ git push origin feature/clases-uml
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 147.82 KiB | 18.48 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Create a pull request for 'feature/clases-uml' on GitHub by visiting:
remote:   https://github.com/emmanuel-6193/Desarrollo_de_SI/pull/new/feature/clases-uml
remote:
To https://github.com/emmanuel-6193/Desarrollo_de_SI.git
 * [new branch]   feature/clases-uml -> feature/clases-uml
$
```

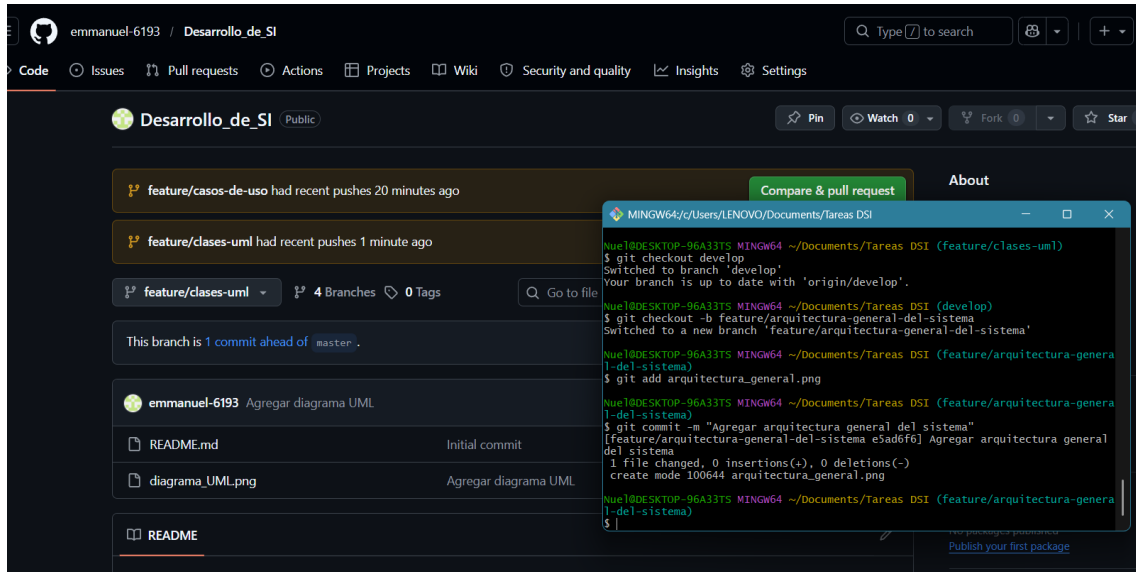
Realizo el comit **git commit -m "Agregar diagrama UML"**



Creo la rama arquitectura general del sistema con el comando **git checkout -b feature/arquitectura-general-del-sistema**

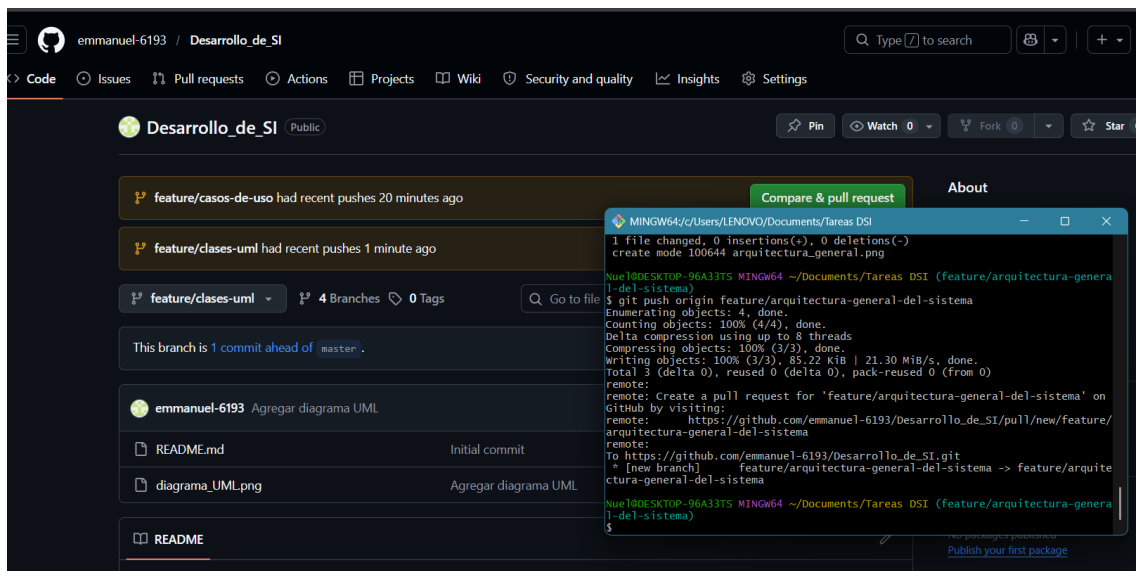


Agrego el diagrama de arquitectura general con el comando **git add arquitectura_general.png**



```
MINGW64~/Users/LENOVO/Documents/Tareas DSI
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
$ git checkout -b feature/arquitectura-general-del-sistema
Switched to a new branch 'feature/arquitectura-general-del-sistema'
$ git add arquitectura_general.png
$ git commit -m "Agregar arquitectura general del sistema"
[feature/arquitectura-general-del-sistema e5ad6f6] Agregar arquitectura general del sistema
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 arquitectura_general.png
$
```

Realizo el comit **git commit -m “Agregar arquitectura general del sistema”**



```
$ git push origin feature/arquitectura-general-del-sistema
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 85.22 KiB | 21.30 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/arquitectura-general-del-sistema' on
remote: GitHub by visiting:
remote: https://github.com/emmanuel-6193/Desarrollo_de_SI/pull/new/feature/arquitectura-general-del-sistema
remote:
remote: https://github.com/emmanuel-6193/Desarrollo_de_SI.git
* [new branch] feature/arquitectura-general-del-sistema -> feature/arquitectura-general-del-sistema
$
```

Y con este comando **git push origin feature/arquitectura-general-del-sistema** queda el diagrama 3 correctamente subido

Justificación del Flujo de Trabajo (Gitflow)

Para esta actividad, decidí implementar el modelo Gitflow porque es una de las mejores prácticas en el desarrollo de software profesional. Mi objetivo fue mantener el repositorio organizado y cumplir con los estándares de calidad exigidos en la guía de la asignatura. He justificado el uso de esta metodología por las siguientes razones:

- Organización por entregables: En lugar de subir todo a la rama principal de una sola vez, utilicé ramas feature/ para cada diagrama (Casos de Uso, UML, Arquitectura). Esto me permitió trabajar en cada parte de forma independiente,



asegurando que cada avance esté bien documentado antes de pasar a la siguiente etapa.

- Control del historial: Gracias al uso de commits específicos, el historial de mi repositorio muestra claramente la evolución del proyecto. Como estudiante, esto me ayuda a demostrar la trazabilidad de mi trabajo y a tener un respaldo ordenado de cada versión de mis diagramas.
- Integración en la rama Develop: Utilicé la rama develop como un espacio de integración. Aquí fui uniendo todos los componentes ya terminados antes de generar la entrega final en la rama master. Esto garantiza que la versión que el docente revisa sea estable y libre de archivos temporales.
- Aprendizaje de estándares industriales: Implementar este flujo me permite familiarizarme con el trabajo en equipo real. Aunque este es un proyecto individual, usar Gitflow demuestra que tengo la capacidad de manejar versiones y colaborar en entornos profesionales donde el orden del código es fundamental.

Historial de ramas/commits

Commits

master

Commits on May 14, 2026

- Eliminar Actividad_Farfan.pdf
emmanuel-6193 committed 3 hours ago
eb97d6d

Commits on May 12, 2026

- commit inicial: subida de la primera version de la presentacion
emmanuel-6193 committed 2 days ago
874d3d0
- Initial commit
emmanuel-6193 authored 2 days ago
Verified 2d059f9

Network graph

Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.

Owners

May

emmanuel-6193

12 14

master

develop

initial commit

Eliminar Actividad_Farfan.pdf

commit inicial: subida de la primera version de la presentacion

emmanuel-6193 / Desarrollo_de_SI

Q Type Z to search

<> Code

 Issues

 Pull requests

 Actions

 Projects

 Wiki

 Security and quality

 Insights

 Settings

Commits

 develop

 All users

 All time

 Commits on May 14, 2026

Eliminar Actividad_Farfan.pdf

 emmanuel-6193 committed 3 hours ago

eb97d6d

 Commits on May 12, 2026

commit inicial: subida de la primera version de la presentacion

 emmanuel-6193 committed 2 days ago

Initial commit

 emmanuel-6193 authored 2 days ago

874d3d0

Verified

2d059f9

emmanuel-6193 / Desarrollo_de_Sl

Q

Type / to search

+

<>

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

Pulse

Contributors

Community

Community standards

Traffic

Commits

Code frequency

Dependency graph

Network

Forks

Actions usage metrics

Network graph

Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.

Owners

May

12

14

emmanuel-6193

main

develop

feature/issue-10

feature/issue-11

feature/issue-12

feature/issue-13

feature/issue-14

feature/issue-15

feature/issue-16

feature/issue-17

feature/issue-18

feature/issue-19

feature/issue-20

feature/issue-21

feature/issue-22

feature/issue-23

feature/issue-24

feature/issue-25

feature/issue-26

feature/issue-27

feature/issue-28

feature/issue-29

feature/issue-30

feature/issue-31

feature/issue-32

feature/issue-33

feature/issue-34

feature/issue-35

feature/issue-36

feature/issue-37

feature/issue-38

feature/issue-39

feature/issue-40

feature/issue-41

feature/issue-42

feature/issue-43

feature/issue-44

feature/issue-45

feature/issue-46

feature/issue-47

feature/issue-48

feature/issue-49

feature/issue-50

feature/issue-51

feature/issue-52

feature/issue-53

feature/issue-54

feature/issue-55

feature/issue-56

feature/issue-57

feature/issue-58

feature/issue-59

feature/issue-60

feature/issue-61

feature/issue-62

feature/issue-63

feature/issue-64

feature/issue-65

feature/issue-66

feature/issue-67

feature/issue-68

feature/issue-69

feature/issue-70

feature/issue-71

feature/issue-72

feature/issue-73

feature/issue-74

feature/issue-75

feature/issue-76

feature/issue-77

feature/issue-78

feature/issue-79

feature/issue-80

feature/issue-81

feature/issue-82

feature/issue-83

feature/issue-84

feature/issue-85

feature/issue-86

feature/issue-87

feature/issue-88

feature/issue-89

feature/issue-90

feature/issue-91

feature/issue-92

feature/issue-93

feature/issue-94

feature/issue-95

feature/issue-96

feature/issue-97

feature/issue-98

feature/issue-99

feature/issue-100

feature/issue-101

feature/issue-102

feature/issue-103

feature/issue-104

feature/issue-105

feature/issue-106

feature/issue-107

feature/issue-108

feature/issue-109

feature/issue-110

feature/issue-111

feature/issue-112

feature/issue-113

feature/issue-114

feature/issue-115

feature/issue-116

feature/issue-117

feature/issue-118

feature/issue-119

feature/issue-120

feature/issue-121

feature/issue-122

feature/issue-123

feature/issue-124

feature/issue-125

feature/issue-126

feature/issue-127

feature/issue-128

feature/issue-129

feature/issue-130

feature/issue-131

feature/issue-132

feature/issue-133

feature/issue-134

feature/issue-135

feature/issue-136

feature/issue-137

feature/issue-138

feature/issue-139

feature/issue-140

feature/issue-141

feature/issue-142

feature/issue-143

feature/issue-144

feature/issue-145

feature/issue-146

feature/issue-147

feature/issue-148

feature/issue-149

feature/issue-150

feature/issue-151

feature/issue-152

feature/issue-153

feature/issue-154

feature/issue-155

feature/issue-156

feature/issue-157

feature/issue-158

feature/issue-159

feature/issue-160

feature/issue-161

feature/issue-162

feature/issue-163

feature/issue-164

feature/issue-165

feature/issue-166

feature/issue-167

feature/issue-168

feature/issue-169

feature/issue-170

feature/issue-171

feature/issue-172

feature/issue-173

feature/issue-174

feature/issue-175

feature/issue-176

feature/issue-177

feature/issue-178

feature/issue-179

feature/issue-180

feature/issue-181

feature/issue-182

feature/issue-183

feature/issue-184

feature/issue-185

feature/issue-186

feature/issue-187

feature/issue-188

feature/issue-189

feature/issue-190

feature/issue-191

feature/issue-192

feature/issue-193

feature/issue-194

feature/issue-195

feature/issue-196

feature/issue-197

feature/issue-198

feature/issue-199

feature/issue-200

feature/issue-201

feature/issue-202

feature/issue-203

feature/issue-204

feature/issue-205

feature/issue-206

feature/issue-207

feature/issue-208

feature/issue-209

feature/issue-210

feature/issue-211

feature/issue-212

feature/issue-213

feature/issue-214

feature/issue-215

feature/issue-216

feature/issue-217

feature/issue-218

feature/issue-219

feature/issue-220

feature/issue-221

feature/issue-222

feature/issue-223

feature/issue-224

feature/issue-225

feature/issue-226

feature/issue-227

feature/issue-228

feature/issue-229

feature/issue-230

feature/issue-231

feature/issue-232

feature/issue-233

feature/issue-234

feature/issue-235

feature/issue-236

feature/issue-237

feature/issue-238

feature/issue-239

feature/issue-240

feature/issue-241

feature/issue-242

feature/issue-243

feature/issue-244

feature/issue-245

feature/issue-246

feature/issue-247

feature/issue-248

feature/issue-249

feature/issue-250

feature/issue-251

feature/issue-252

feature/issue-253

feature/issue-254

feature/issue-255

feature/issue-256

feature/issue-257

feature/issue-258

feature/issue-259

feature/issue-260

feature/issue-261

feature/issue-262

feature/issue-263

feature/issue-264

feature/issue-265

feature/issue-266

feature/issue-267

feature/issue-268

feature/issue-269

feature/issue-270

feature/issue-271

feature/issue-272

feature/issue-273

feature/issue-274

feature/issue-275

feature/issue-276

feature/issue-277

feature/issue-278

feature/issue-279

feature/issue-280

feature/issue-281

feature/issue-282

feature/issue-283

feature/issue-284

feature/issue-285

feature/issue-286

feature/issue-287

feature/issue-288

feature/issue-289

feature/issue-290

feature/issue-291

feature/issue-292

feature/issue-293

feature/issue-294

feature/issue-295

feature/issue-296</

emmanuel-6193 / Desarrollo_de_SI

🔍

Type / to search

+

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

Commits

feature/arquitect...

All users

All time

Commits on May 14, 2026

Agregar arquitectura general del sistema

emmanuel-6193 committed 1 hour ago

e5ad6f6

Eliminar Actividad_Farfan.pdf

emmanuel-6193 committed 3 hours ago

eb97d6d

Commits on May 12, 2026

commit inicial: subida de la primera version de la presentacion

emmanuel-6193 committed 2 days ago

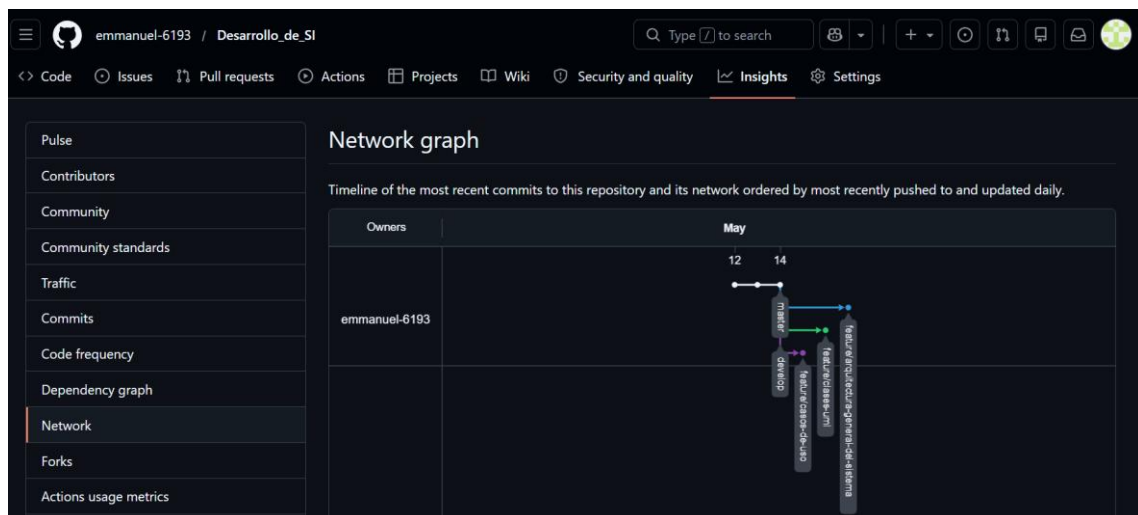
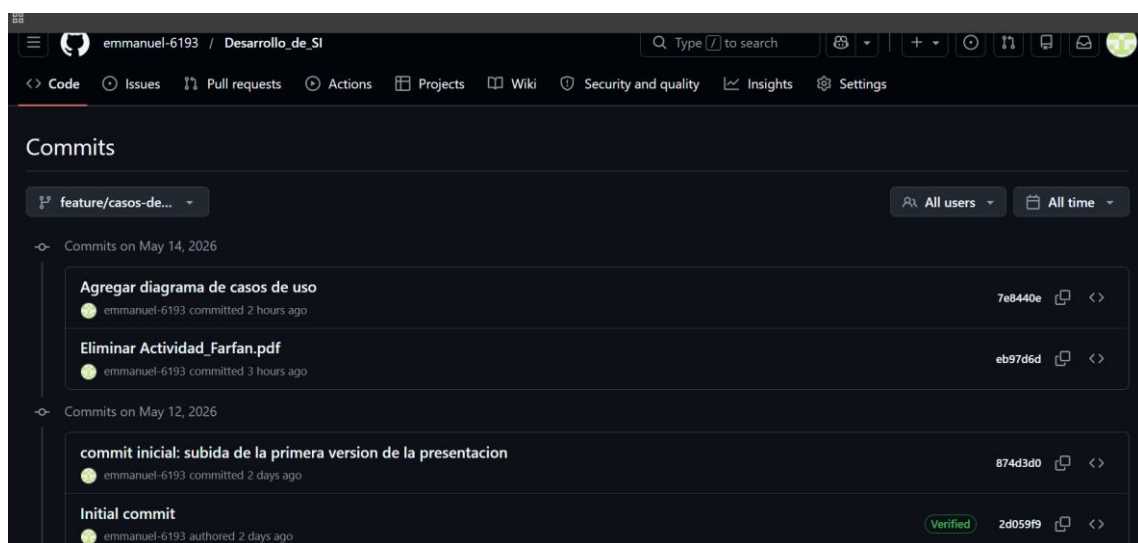
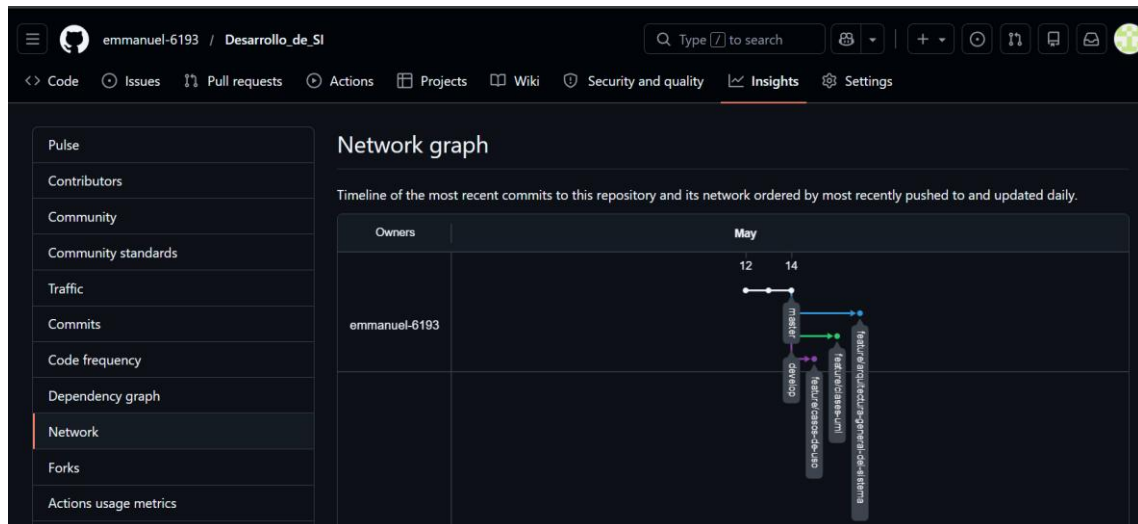
874d3d0

Initial commit

emmanuel-6193 authored 2 days ago

2d059f9

Verified





emmanuel-6193 / Desarrollo_de_Si

Type to search

+

+

+

+

+

+

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

Pulse

Contributors

Community

Community standards

Traffic

Commits

Code frequency

Dependency graph

Network

Forks

Actions usage metrics

Actions performance metrics

Network graph

Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.

Owners	May
emmanuel-6193	